# Agenda

01 Overview

02 Kubernetes Security

03 Pod Security

04 Other Tooling to help

05 Managed Kubernetes (Cloud Offering)

# What am I expecting from you (audience)?

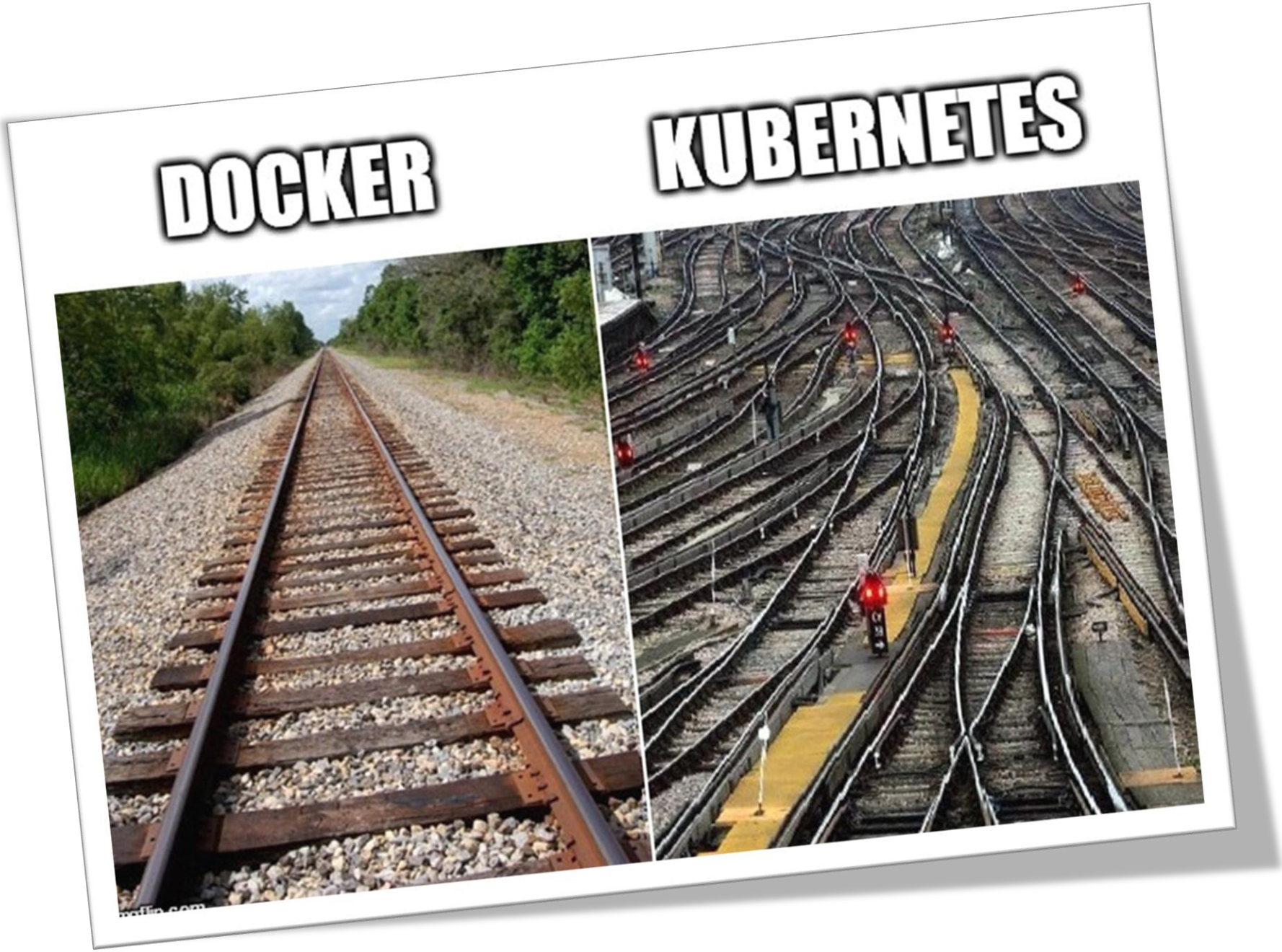
CURIOUS AND WILLING TO LEARN NEW THINGS

FAMILIAR WITH LINUX ERR...I MEAN GNU/LINUX

FAMILIAR WITH NETWORK OR APPLICATION SECURITY

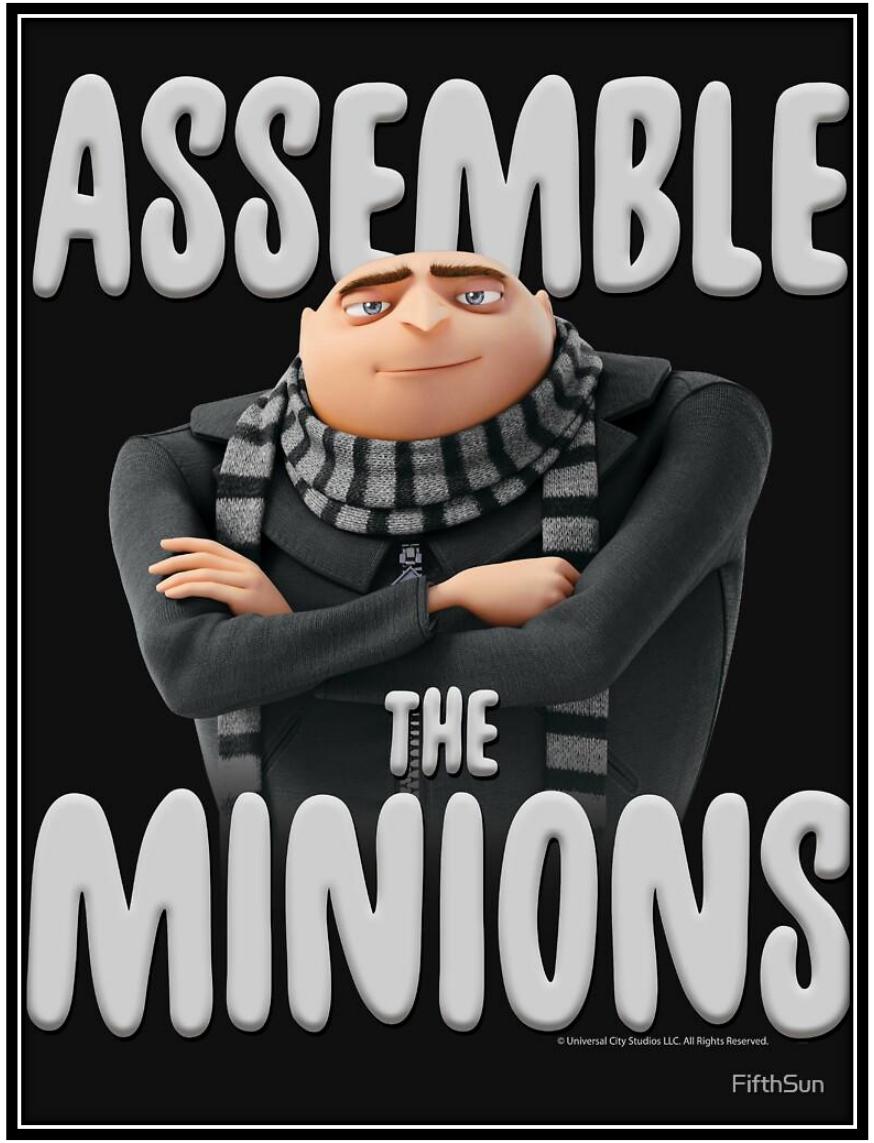
FAMILIAR WITH BASIC VULNERABILITY AND EXPLOIT TERMINOLOGY

FAMILIAR WITH VULNERABILITY AND PENETRATION TESTING


DOCKER
KUBERNETES

# WILL TALK ABOUT:

- Scan containers and pods for vulnerabilities or misconfigurations.
- Run Containers and pods with the least privileges possible.
- Use network separation to control the amount of damage a compromise can cause.
- Use firewalls to limit unneeded network connectivity and encryption to protect confidentiality.
- Use strong authentication and authorization to limit user and administrator access as well as to limit the attack surface.
- Periodically review all Kubernetes settings and use vulnerability scans to help ensure risks are appropriately accounted for and security patches are applied.



# WHAT IS A CONTAINER?

Containers are a technology for packaging the (compiled) code for an application along with the dependencies it needs at run time.

Each container you run is repeatable; the standardization of having dependencies included means you get the same behavior wherever you run it.

Think of a container as "Code + Config + Runtime" packaged in an archive, stored locally or in a Git-like remote repository called a container Registry.



# WHAT IS KUBERNETES?

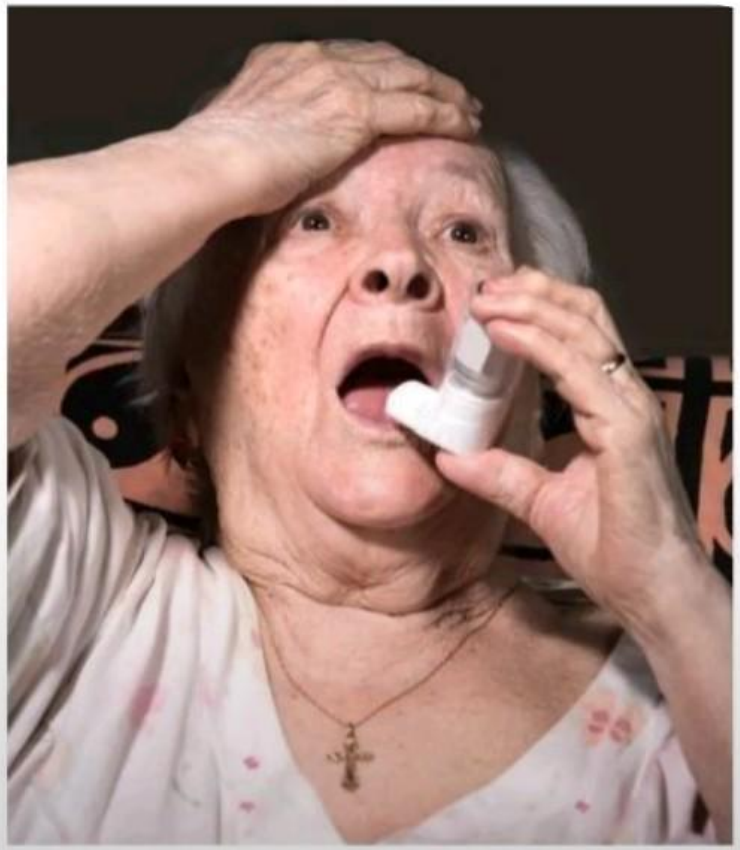
Kubernetes is a portable, extensible, open-source platform for managing containerized workloads and services that facilitates both declarative configuration and automation.



Jennifer Aniston at 51 years old



Kubernetes Engineers in their 30s

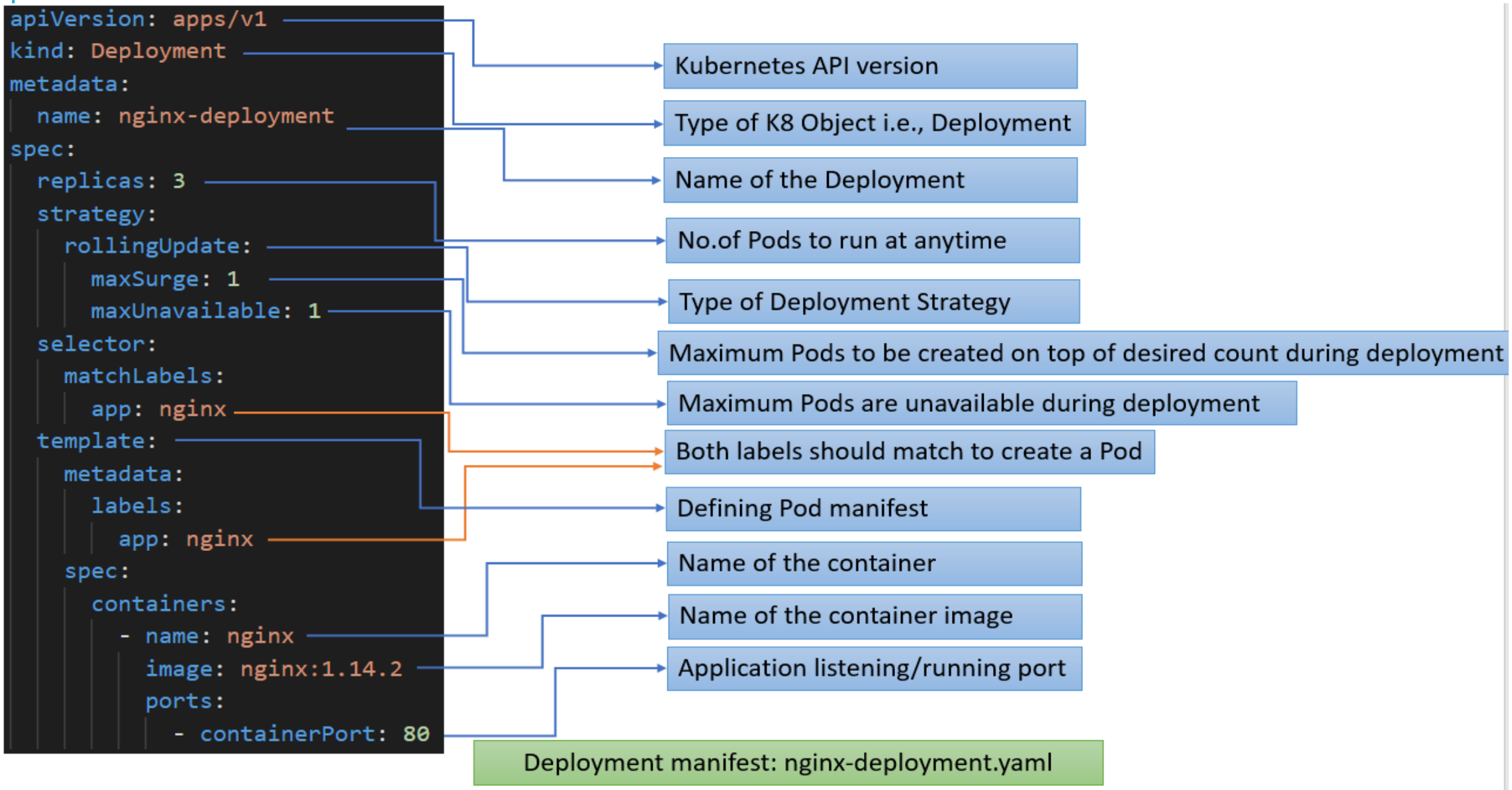


# OTHER KEY RESOURCES

- Namespace
- Replica Set
- Deployment
- ConfigMap
- Secret (Encoded, not encrypted, by default)
- Volume
- Persistent Volume (in Kubernetes is a piece of storage in the cluster that has a lifecycle independent of any individual pod that uses it)
- Persistent Volume Claim (in Kubernetes is a request for storage resources made by a user or application)
- Ingress



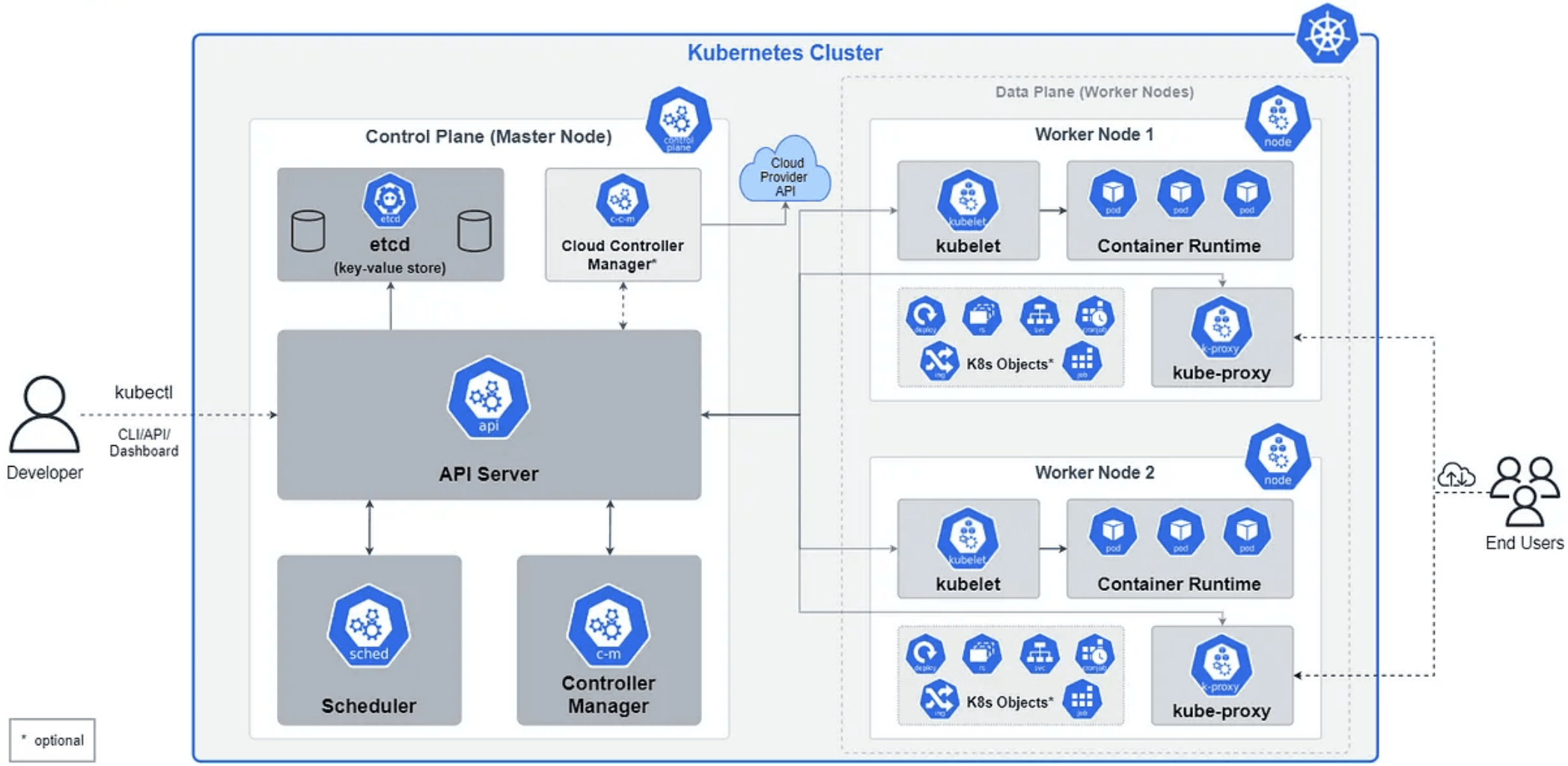
# OTHER KEY RESOURCES



# KUBERNETES THREAT MODEL



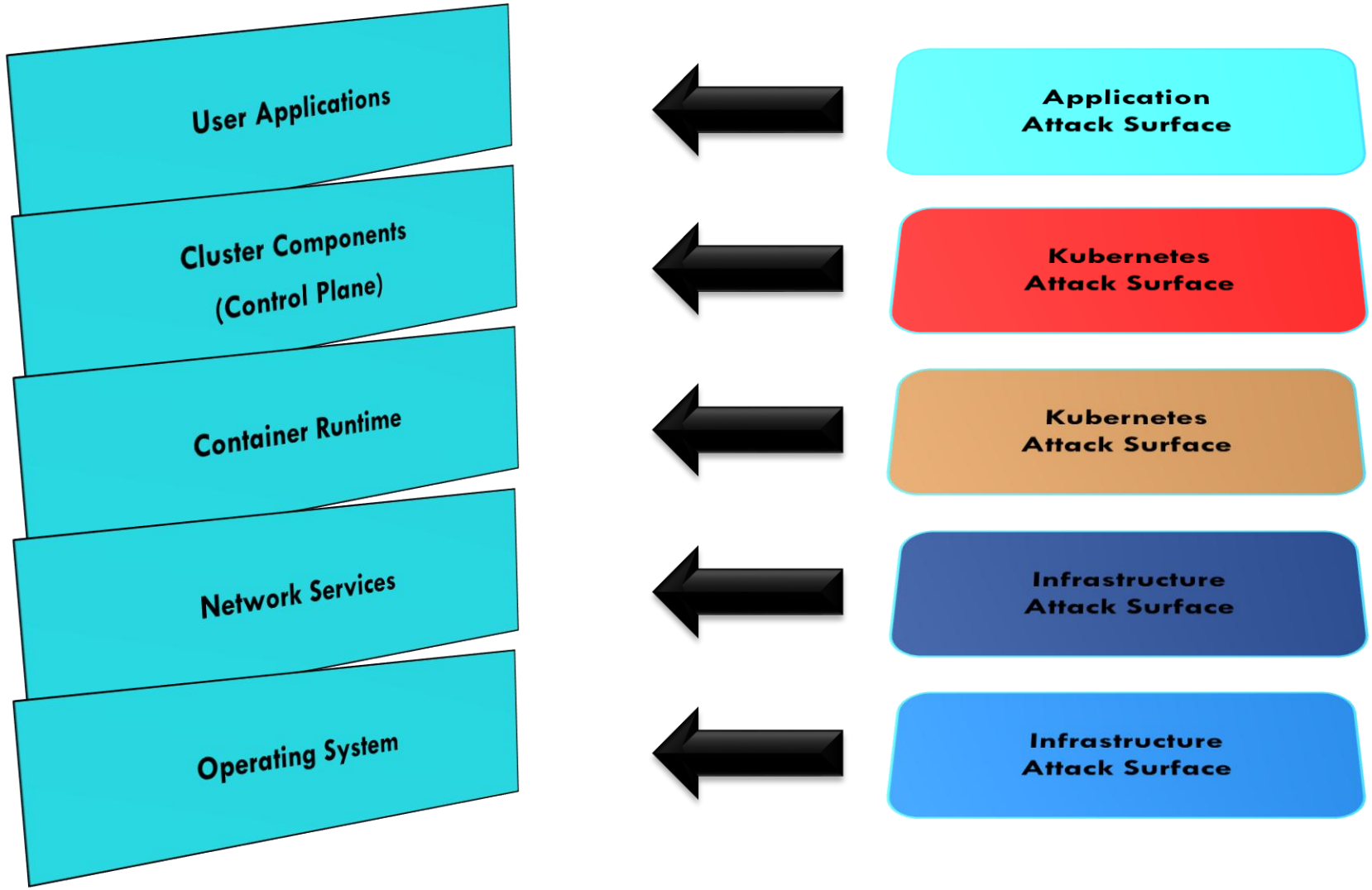
# KUBERNETES COMPONENTS



# A SIMPLE THREAT MODEL

| | |
|---|---|
| 01 Who are the attackers? | • External (from the internet)<br>• Internal (attacker in a pod)<br>• Developer (User with some access in the cluster)<br>• Malicious Administrator<br>• End User |
| 02 What can they attack? | • Cluster state storage (etcd)<br>• Secrets<br>• Volume (Data Breach)<br>• Container Image (Private Repository)<br>• Compute Resources (Example: Crypto Mining) |
| 03 How can they attack? | • Level Configuration Weakness<br>• Exploit vulnerabilities<br>• Exploit trust across components<br>• Lack of appropriate AuthZ controls<br>• Lack of Security hardening of the cluster |

# KUBERNETES CLUSTER ATTACK SURFACE



# KUBERNETES ATTACKS IN RECENT YEARS

# KINSING HACKER GROUP EXPLOITS MORE FLAWS TO EXPAND BOTNET FOR CRYPTOJACKING

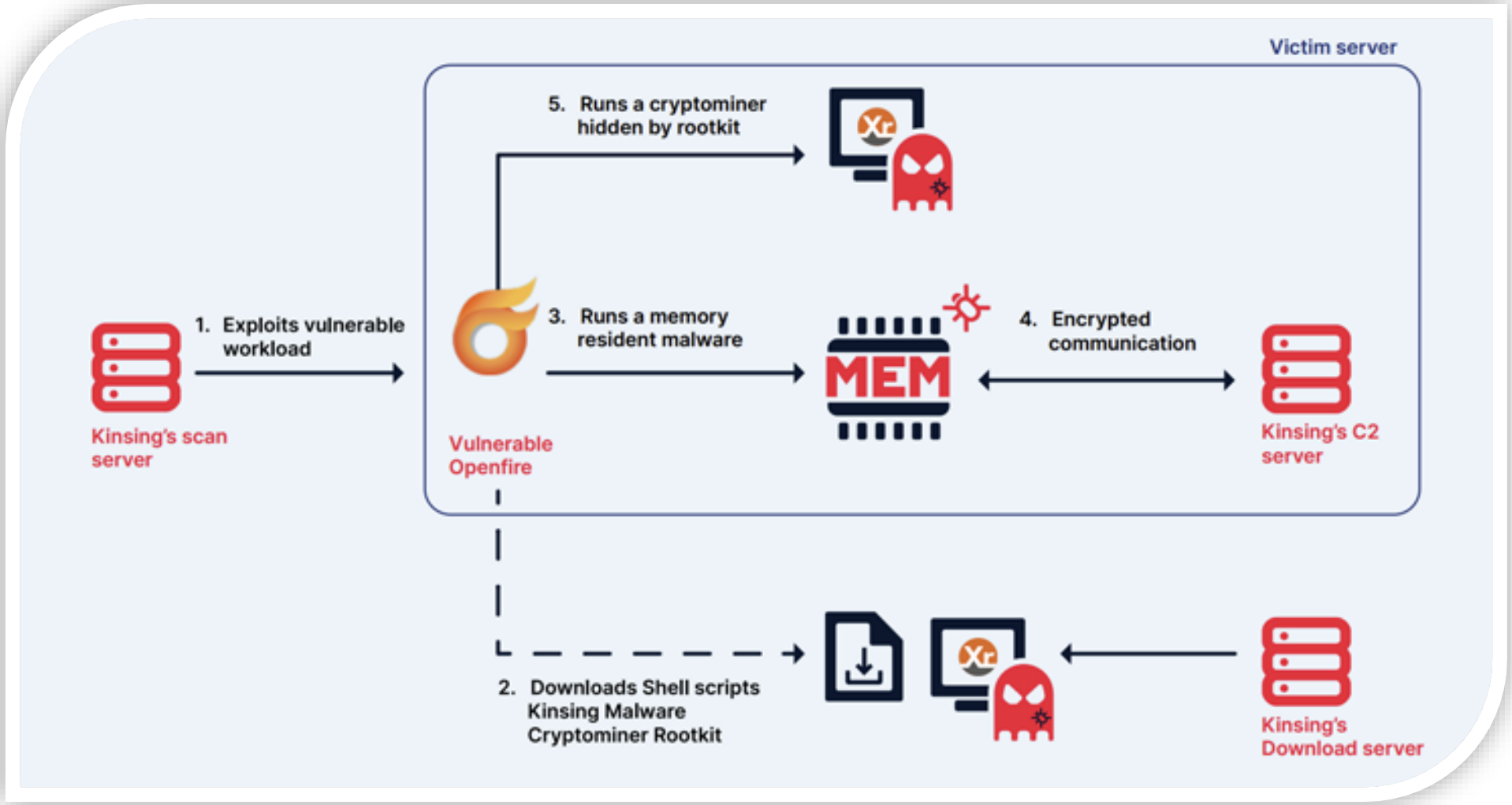
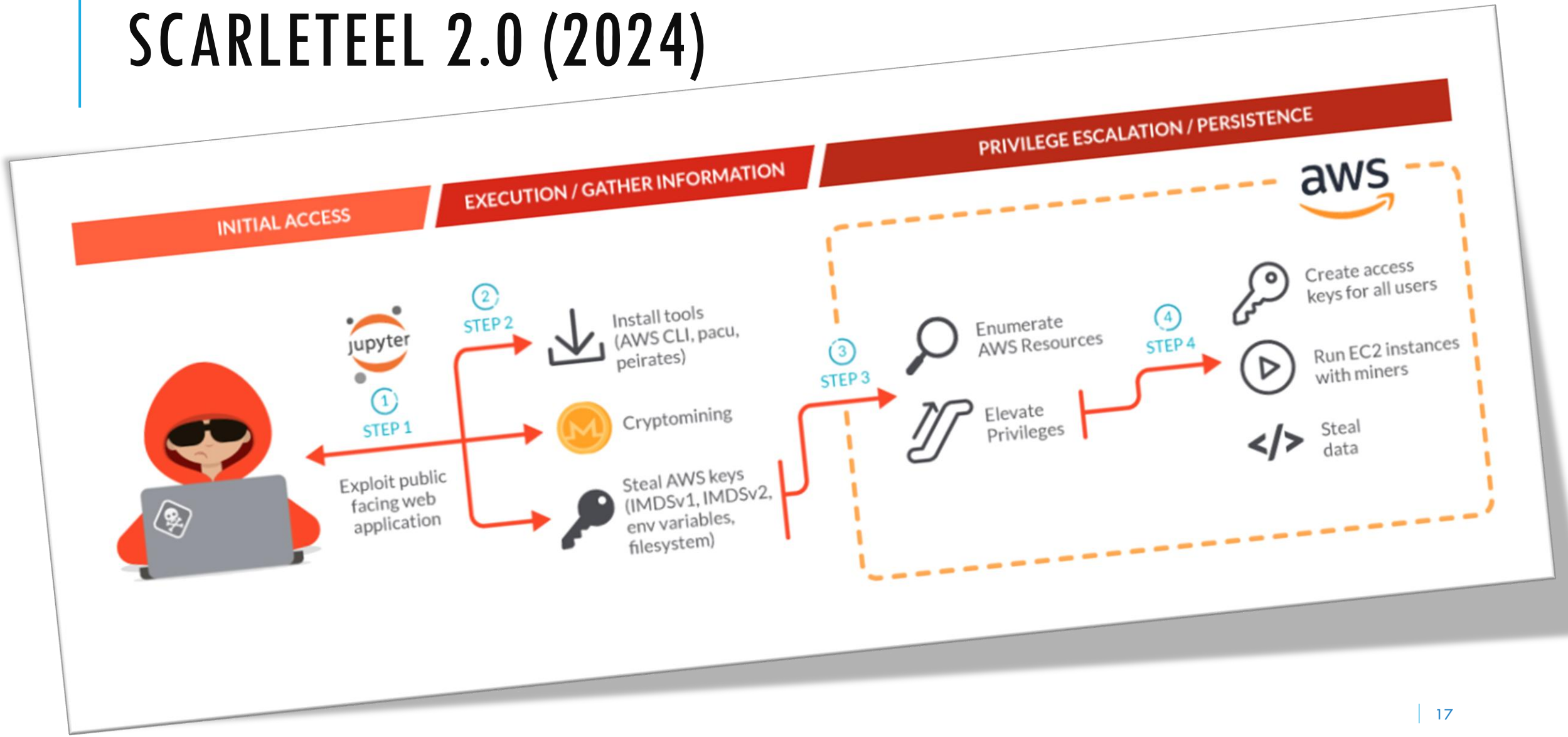


*Table 1 – Vulnerabilities Exploited by Kinsing*

| Vulnerability | Description |
|---|---|
| Misconfigured Remote Docker API | Docker APIs that are open to the Internet with weak password or without authentication |
| Misconfigured Redis Server | Redis server with weak password or without password protection |
| CVE-2020-25213 | WordPress File Manager Plugin RCE |
| CVE-2020-23814 | XXL-JOB RCE |
| CVE-2020-11651 and CVE-2020-11652 | SaltStack RCE |
| CVE-2019-3396 | Atlassian Confluence Widget Connector Macro Velocity Template Injection |
| CVE-2019-0193 | RCE via DataImportHandler |
| CVE-2018-20062 | NoneCMS ThinkPHP RCE |
| CVE-2017-15718 | Apache Hadoop YARN NodeManager vulnerability |
| CVE-2017-11610 | Supervisord RCE |
| CVE-2017-9841 | PHPUnit RCE |

# SCARLETEEL 2.0 (2024)



# TECHNIQUE: A HIGHLY SOPHISTICATED OPERATION TARGETING AWS EC2 AND OTHER KUBERNETES INFRASTRUCTURE.

**Attack Chain:**

- Scripts are aware of being in a Fargate-hosted container and can collect credentials.
- Escalation to Admin in the victim's AWS account and spin up EC2 instances running miners.
- Tools and techniques improved to expand their attack capabilities and their defense evasion techniques.
- Attempted exploitation of IMDSv2 to retrieve the token and then use it to retrieve the AWS credentials.
- Changes in C2 domains multiple times, including utilizing public services used to send and retrieve data.
- Using AWS CLI and pacu on the exploited containers to further exploit AWS.
- Using peirates to further exploit Kubernetes.

**Key Lesson:** It demonstrated the critical risk of **container-to-Kubernetes lateral movement,** where a container compromise leads to a full Kubernetes account breach.

# SCRUTINY WEAPONIZED FOR SILENT CRYPTOJACKING

- **Attackers are now using legitimate open-source security tools like kubeaudit and kubescape against victims.**
- **They run these tools inside compromised clusters to identify misconfigurations that they themselves can exploit to deploy hidden, low-resource-consuming cryptojacking containers that evade typical monitoring.**

## Scrutiny Weaponized for Silent Cryptojacking

- **Attackers upload malicious container images to public repositories (like Docker Hub) or helm charts with embedded malware (crypto miners, credential stealers).**
- **Developers who pull these images without proper vetting inadvertently compromise their clusters.**

# COMMON INITIAL ATTACK VECTORS (HOW THEY GET IN)

**Misconfigurations:** The #1 cause. This includes:

- Exposed Kubernetes API server to the internet.
- Overly permissive RBAC roles (especially cluster-admin bound to default service accounts).
- Secrets stored in plaintext or in environment variables.
- Running containers with privileged: true or as root.

**Unpatched Software:** Failing to update Kubernetes components (kube-apiserver, kubelet), container runtimes (like runc vulnerabilities CVE-2024-21626), or workloads.

**Compromised Credentials:** Stolen Kubernetes provider IAM keys, Kubernetes service account tokens, or developer CI/CD credentials.

**Vulnerable Applications:** Web applications in containers with common vulnerabilities (SQLi, RCE) that provide an initial foothold.


ENOUGH TALK
SHOW ME A DEMO!!!

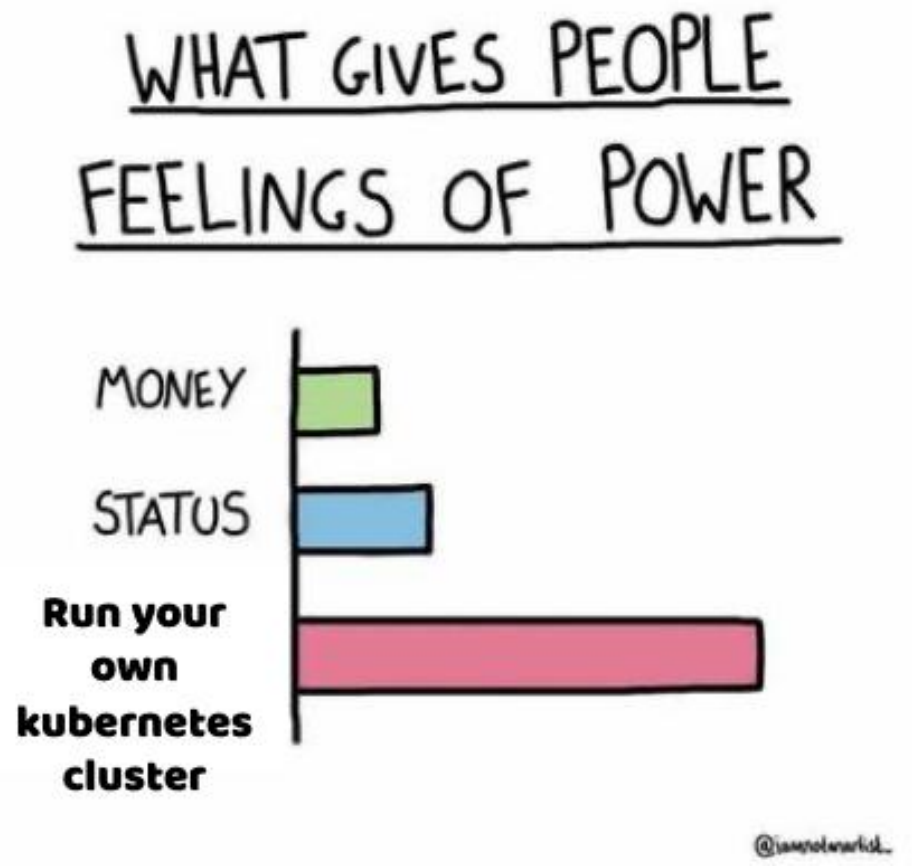
# KUBERNETES HARDENING / SECURITY

# KUBERNETES HARDENING / SECURITY

**Remember: Follow the "Principle of least privilege."**

## Next to Discuss

- **Pod Security Policy**
- **Open Policy Agent**
- **Network Policy**
- **Secrets**
- **Securing The Cluster**



# POD SECURITY POLICY

PSP is one way to control the security-related attributes of pods, **including container privilege** levels.

- Do not run application processes as root
- Do not allow privilege escalation
- Use a read-only root filesystem
- Use the default (masked) /proc filesystem mount
- Do not use the host network or process space
- Drop unused and unnecessary Linux capabilities
- Service Account control

# POD SECURITY POLICY

policy/example-psp.yaml

```yaml
apiVersion: policy/v1beta1
kind: PodSecurityPolicy
metadata:
  name: example
spec:
  privileged: false  # Don't allow privileged pods!
  # The rest fills in some required fields.
  seLinux:
    rule: RunAsAny
  supplementalGroups:
    rule: RunAsAny
  runAsUser:
    rule: RunAsAny
  fsGroup:
    rule: RunAsAny
  volumes:
  - '*'
```

When a PSP resource is created, **it does nothing**. You need to authorize using RBAC!

# OPEN POLICY AGENT {GATEKEEPER}

I need to control other pod fields or any fields in other resources? How can I achieve that?

**The answer is:**



**Gatekeeper controller provides you ability to:**

- Required labels
- Required resources section
- Mutate container images to always point to the corporate image registry.
- Set node and pod affinity and anti-affinity selectors to Deployments
- You can enforce anything that you want to see or not to see in configs

# OPEN POLICY AGENT {GATEKEEPER}

constraint.yaml

```yaml
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sAllowedRepos
metadata:
  name: repo-constraint
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
    namespaces:
      - "default"
  parameters:
    repos:
      - "openpolicyagent/"
      - "myregistry.com/"
```

**Example: Enforce to use only allowed container registries.**

# OPEN POLICY AGENT {GATEKEEPER}

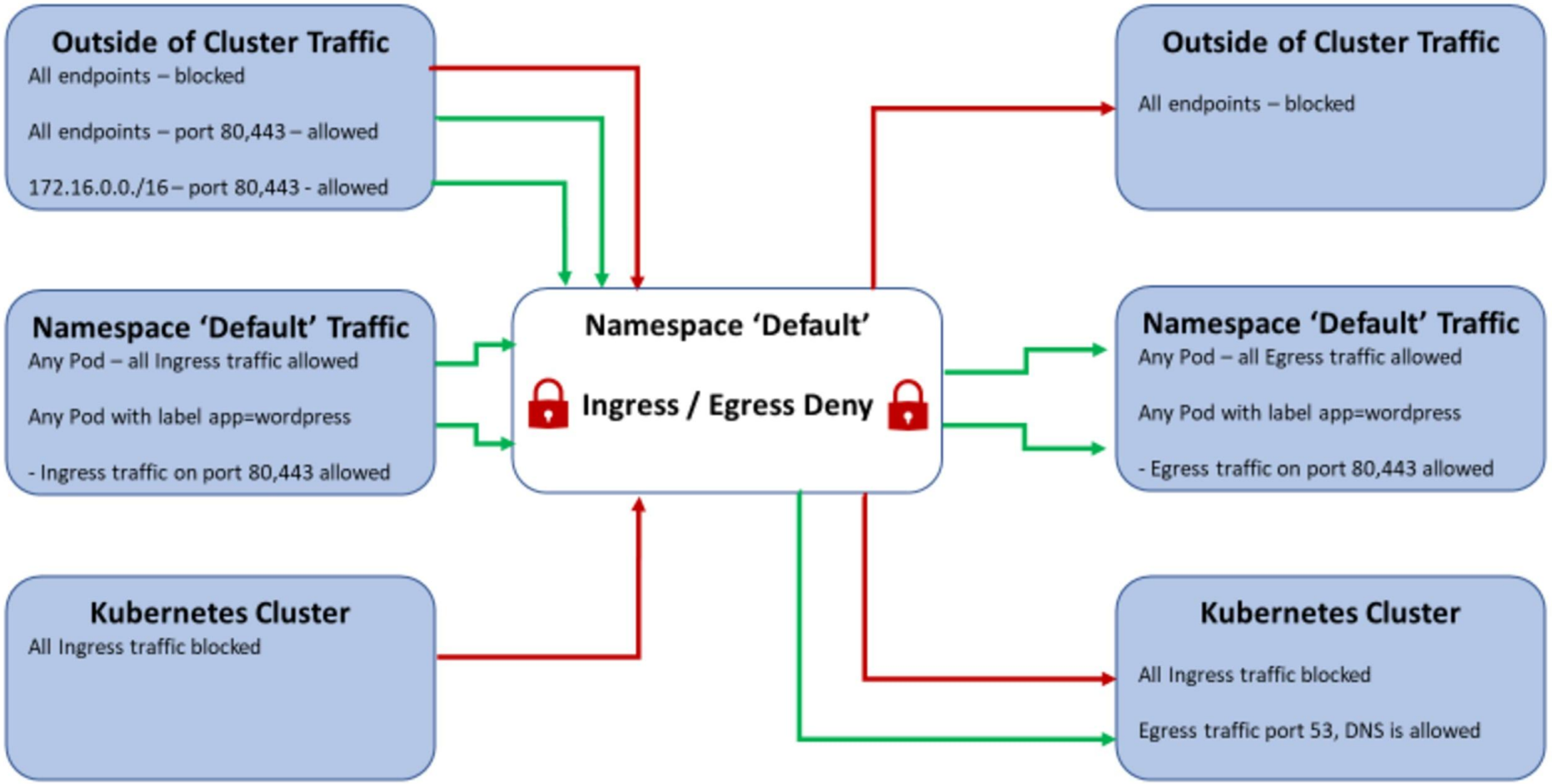


User

```
1 > kubectl apply -f template.yaml
constrainttemplate.templates.gatekeeper.sh/k8sallowedrepos created
> kubectl apply -f constraint.yaml
k8sallowedrepos.constraints.gatekeeper.sh/repo-constraint created

2 > kubectl run my-pod --image=nginx
Error from server (Forbidden): admission webhook
"validation.gatekeeper.sh" denied the request: [repo-constraint]
container <my-pod> has an invalid image repo <nginx>, allowed repos
are ["openpolicyagent/", "myregistry.com/"]
```

# NETWORK POLICY



# NETWORK POLICY

```
C:\>kubectl apply -f sample-network-policy.yaml
networkpolicy.networking.k8s.io/sample-network-policy configured

C:\>kubectl describe networkpolicy sample-network-policy
Name:         sample-network-policy
Namespace:    default
Created on:   2022-10-30 18:16:43 -0700 PDT
Labels:       <none>
Annotations:  <none>
Spec:
  PodSelector:     app=webapp
  Allowing ingress traffic:
    To Port: 443/TCP
    To Port: 80/TCP
    From:
      PodSelector: app=webapp
    ----------
    To Port: <any> (traffic allowed to all ports)
    From:
      PodSelector: <none>
    ----------
    To Port: 443/TCP
    To Port: 80/TCP
    From:
      IPBlock:
        CIDR: 172.16.0.0/16
        Except:
    ----------
    To Port: 443/TCP
    To Port: 80/TCP
    From:
      IPBlock:
        CIDR: 0.0.0.0/0
        Except:
```

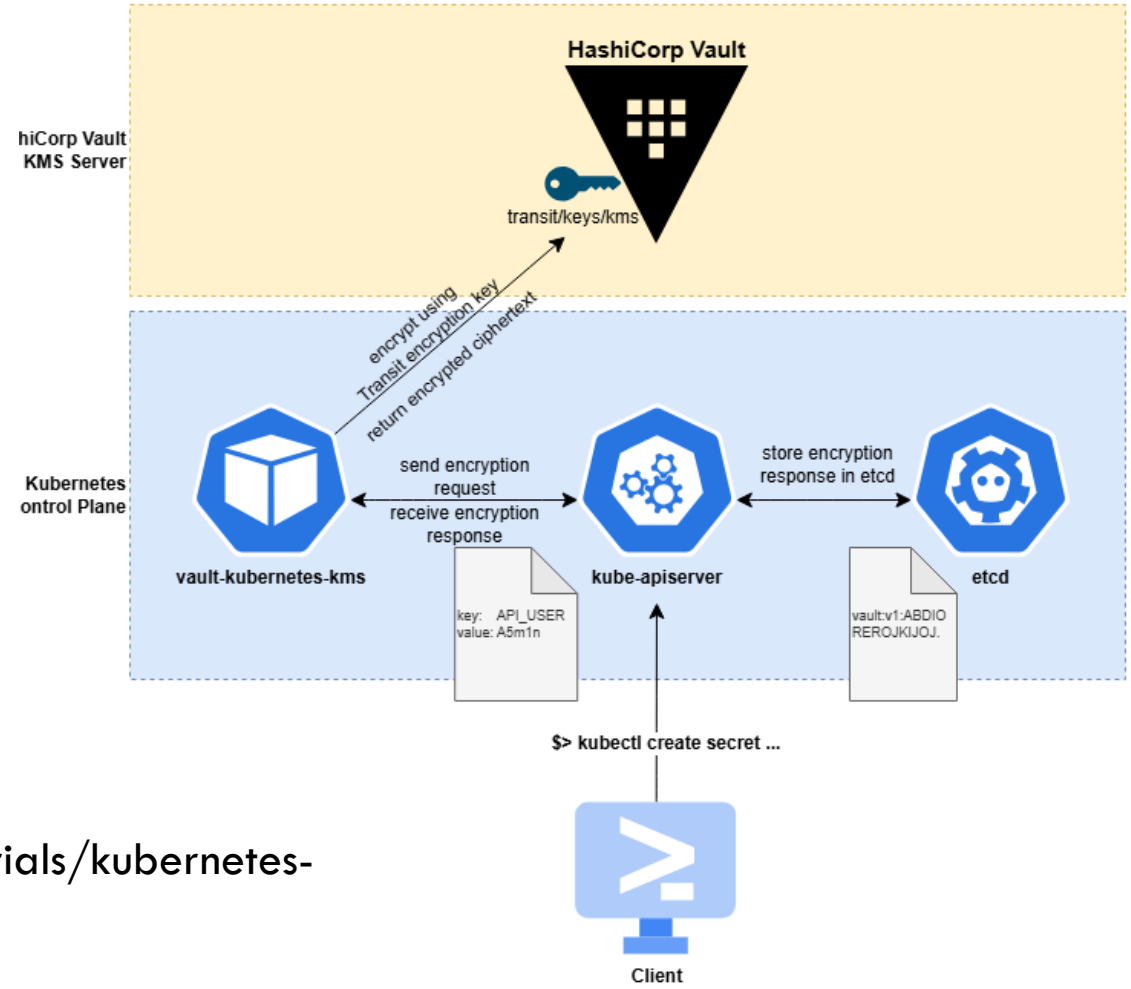
```
  Allowing egress traffic:
    To Port: 53/UDP
    To:
      NamespaceSelector: <none>
      PodSelector: k8s-app=kube-dns
    ----------
    To Port: <any> (traffic allowed to all ports)
    To:
      PodSelector: <none>
    ----------
    To Port: 443/TCP
    To Port: 80/TCP
    To:
      PodSelector: app=webapp
  Policy Types: Egress, Ingress
```

# SECRETS

**Where do I version control my secrets?**

**What is the solution for this?**

- Paper /USB/CDR two fireproof safes?
- Vault (or actually Consul)




https://developer.hashicorp.com/vault/tutorials/kubernetes-introduction/vault-secrets-operator

# SECURING THE CLUSTER

**API Server**

**By default, the API server will listen on what is rightfully port 8080**

- *Close the insecure port* by arg in API server's –insecure-port flag to 0.
- And –insecure-bind-address is not set.

- Any requests to this port *bypass* authentication and authorization checks



# SECURING THE CLUSTER

**etcd**

**The etcd backend database is a *critical component* and needs to be secured within the cluster.**

- *Close the insecure port* by arg in API server's –insecure-port flag to 0.
- And –insecure-bind-address is not set.

- Etcd server should be configured to *only trust certificates* assigned to API servers.



--authorization-mode=Webhook

# SECURING THE CLUSTER

**Kubelet**

**The kubelet is the agent that is responsible for launching pods (not Check these parameters:**

- *Disable anonymous access with –anonymous-auth=false*
- Ensure that requests are authorized by setting
- --authorization-mode=Webhook \
- --client-ca-file=/etc/kubernetes/pki/ca.crt

other than Alwaysallow.



--authorization-mode=Webhook

# SECURING THE CLUSTER

## Kubernetes Dashboard

**The Dashboard has historically been used by attackers to gain Kubernetes cluster.**

**Check these parameters:**

- Allow only authenticated access. Only known users should be able to access the Dashboard.
- Use RBAC. Limit the privileges that users have so they can they need to.
- Don't expose your dashboard to the public internet.
- Unless you really know what, you're doing.



--authorization-mode=Webhook

# SECURING THE CLUSTER

**The Following table lists the control plane ports and services.**

**You can try to curl on each port and check if it's secure.**

| Protocol | Direction | Port Range | Purpose |
|---|---|---|---|
| TCP | Inbound | 6443 or 8080 if not disabled | Kubernetes API Server |
| TCP | Inbound | 2379, 2380 | etcd server client API |
| TCP | Inbound | 10250 | Kubelet API |
| TCP | Inbound | 10251 | Kuve-Scheduler |
| TCP | Inbound | 10252 | Kube-controller-manager |
| TCP | Inbound | 10258 | Kubernetes-controller-manager(Optional) |



# OTHER TOOLING TO HELP

# TOOLS

- Trivy
- Grype
- Kics
- Kubescape
- Kubectl
- kubiscan
- Kube hunter
- Kube bench
- kubeaudit
- kubescore
- checkov
- kubesec
- kubesploit



--authorization-mode=Webhook

# KUBESCAPE

## Kubescape is the first tool for testing if Kubernetes is deployed securely.

```
✅  Initialized scanner
✅  Loaded policies
✅  Loaded exceptions
✅  Loaded account configurations
✅  Accessed Kubernetes objects
✅  Collected RBAC resources
Control: C-0068 100% |████████████████████████████████████████████████████████████████████| (24/24, 84 it/s)
✅  Done scanning. Cluster: docker-desktop
✅  Done aggregating results

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

Controls: 24 (Failed: 12, Passed: 10, Action Required: 2)
Failed Resources by Severity: Critical — 0, High — 4, Medium — 14, Low — 3
```

| SEVERITY | CONTROL NAME | FAILED RESOURCES | ALL RESOURCES | % COMPLIANCE-SCORE |
|---|---|---|---|---|
| Critical | Disable anonymous access to Kubelet service | 0 | 0 | Action Required * |
| Critical | Enforce Kubelet client TLS authentication | 0 | 0 | Action Required * |
| High | Resource limits | 3 | 21 | 86% |
| High | Host PID/IPC privileges | 1 | 21 | 95% |
| Medium | Non-root containers | 2 | 21 | 90% |
| Medium | Allow privilege escalation | 2 | 21 | 90% |
| Medium | Ingress and Egress blocked | 3 | 21 | 86% |
| Medium | Automatic mapping of service account | 2 | 68 | 97% |
| Medium | Cluster internal networking | 1 | 5 | 80% |
| Medium | Linux hardening | 2 | 21 | 90% |
| Medium | Secret/ETCD encryption enabled | 1 | 1 | 0% |
| Medium | Audit logs enabled | 1 | 1 | 0% |
| Low | Immutable container filesystem | 2 | 21 | 90% |
| Low | PSP enabled | 1 | 1 | 0% |
| | RESOURCE SUMMARY | 8 | 176 | 75.23% |

```
FRAMEWORK NSA
```

--authorization-mode=Webhook

# KUBE-BENCH

**The same tool as kubescape, but could be deployed as a CronJob and executed on a regular basis.**

```
[INFO] 1 Master Node Security Configuration
[INFO] 1.1 API Server
[FAIL] 1.1.1 Ensure that the --allow-privileged argument is set to false (Scored)
[FAIL] 1.1.2 Ensure that the --anonymous-auth argument is set to false (Scored)
[PASS] 1.1.3 Ensure that the --basic-auth-file argument is not set (Scored)
[PASS] 1.1.4 Ensure that the --insecure-allow-any-token argument is not set (Scored)
[FAIL] 1.1.5 Ensure that the --kubelet-https argument is set to true (Scored)
[PASS] 1.1.6 Ensure that the --insecure-bind-address argument is not set (Scored)
[PASS] 1.1.7 Ensure that the --insecure-port argument is set to 0 (Scored)
[PASS] 1.1.8 Ensure that the --secure-port argument is not set to 0 (Scored)
[FAIL] 1.1.9 Ensure that the --profiling argument is set to false (Scored)
[FAIL] 1.1.10 Ensure that the --repair-malformed-updates argument is set to false (Scored)
[PASS] 1.1.11 Ensure that the admission control policy is not set to AlwaysAdmit (Scored)
[FAIL] 1.1.12 Ensure that the admission control policy is set to AlwaysPullImages (Scored)
[FAIL] 1.1.13 Ensure that the admission control policy is set to DenyEscalatingExec (Scored)
[FAIL] 1.1.14 Ensure that the admission control policy is set to SecurityContextDeny (Scored)
[PASS] 1.1.15 Ensure that the admission control policy is set to NamespaceLifecycle (Scored)
[FAIL] 1.1.16 Ensure that the --audit-log-path argument is set as appropriate (Scored)
[FAIL] 1.1.17 Ensure that the --audit-log-maxage argument is set to 30 or as appropriate (Scored)
[FAIL] 1.1.18 Ensure that the --audit-log-maxbackup argument is set to 10 or as appropriate (Scored)
[FAIL] 1.1.19 Ensure that the --audit-log-maxsize argument is set to 100 or as appropriate (Scored)
[PASS] 1.1.20 Ensure that the --authorization-mode argument is not set to AlwaysAllow (Scored)
[PASS] 1.1.21 Ensure that the --token-auth-file parameter is not set (Scored)
[FAIL] 1.1.22 Ensure that the --kubelet-certificate-authority argument is set as appropriate (Scored)
```

--authorization-mode=Webhook

# KUBESEC

**Kubectl plugin for scanning Kubernetes pods, deployments, daemonsets, and statefulsets.**

**Suggests what should be *improved or changed*.**

```
[
  {
    "object": "Pod/security-context-demo.default",
    "valid": true,
    "message": "Failed with a score of -30 points",
    "score": -30,
    "scoring": {
      "critical": [
        {
          "selector": "containers[] .securityContext .capabilities .add == SYS_ADMIN",
          "reason": "CAP_SYS_ADMIN is the most privileged capability and should always be
avoided"
        }
      ],
      "advise": [
        {
          "selector": "containers[] .securityContext .runAsNonRoot == true",
          "reason": "Force the running image to run as a non-root user to ensure least
privilege"
        },
        {
          // ...
        }
      ]
    }
  }
]
```

--authorization-mode=Webhook

# KUBEAUDIT

**Kubeaudit is a command-line tool to audit a Kubernetes cluster for various security concerns, such as:**

- **Run as non-root**
- **Use a read-only root filesystem**
- **Drop scary capabilities, don't add new ones**
- **Don't run privileged**

```
$:> kubectl run kubeaudit --image shopify/kubeaudit  --rm -it
If you don't see a command prompt, try pressing enter.
W1124 15:12:58.786627       1 warnings.go:70] extensions/v1beta1 Ingress is deprecated in v1.14+, unavailable in v1.22+; use networking.k8s.io/v1 Ingress
W1124 15:12:59.185569       1 warnings.go:70] policy/v1beta1 PodSecurityPolicy is deprecated in v1.21+, unavailable in v1.25+

---------------- Results for ---------------

  apiVersion: v1
  kind: Pod
  metadata:
    name: kubeaudit
    namespace: default

----------------------------------------

-- [error] AppArmorAnnotationMissing
   Message: AppArmor annotation missing. The annotation 'container.apparmor.security.beta.kubernetes.io/kubeaudit' should be added.
   Metadata:
      Container: kubeaudit
      MissingAnnotation: container.apparmor.security.beta.kubernetes.io/kubeaudit

-- [error] AutomountServiceAccountTokenTrueAndDefaultSA
   Message: Default service account with token mounted. automountServiceAccountToken should be set to 'false' on either the ServiceAccount or on the PodSpec
should be used.

-- [error] CapabilityOrSecurityContextMissing
   Message: Security Context not set. The Security Context should be specified and all Capabilities should be dropped by setting the Drop list to ALL.
   Metadata:
      Container: kubeaudit
```

# KICS

## keeping infrastructure as code secure is an open-source solution for static code analysis of Infrastructure as Code

```
Running with gitlab-runner 16.9.1 (782c6ecb)
  on green-4.saas-linux-small-amd64.runners-manager.gitlab.com/default ntHFEtyX, system ID: s_8990de21c550
  feature flags: FF_USE_IMPROVED_URL_MASKING:true
Resolving secrets                                                
```


BRICK
BRICK
QUESTIONS!


THANK YOU FOR YOUR TIME.